Solution to Problem 3 in Problem Sheet - Week 5

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm

plt.style.use("ggplot")
```

Part (a)

In the case $q = 2$, the weights satisfy $\pi_2 = 1 - \pi_1$. Hence, we need to keep track of π_1 only. The following function implements the EM algorithm with $d = 1$, outputting the histories of estimates of π_1, μ_1, σ_1, μ_2 and σ_2, respectively, as a tuple.

In [2]:
```
def gaussian_mixture_em(x, n_iter=100):
    pi1 = np.empty(n_iter + 1)
    mu1 = np.empty(n_iter + 1)
    sigma1 = np.empty(n_iter + 1)
    mu2 = np.empty(n_iter + 1)
    sigma2 = np.empty(n_iter + 1)

    pi1[0] = 0.5
    # Initial means cannot be equal, so we use the 25th and 75th percentiles:
    mu1[0], mu2[0] = np.quantile(x, [0.25, 0.75])
    sigma1[0] = sigma2[0] = np.std(x)

    for i in range(n_iter):
        # Expectation:
        resp = pi1[i] * norm.pdf(x, mu1[i], sigma1[i])
        resp /= resp + (1 - pi1[i]) * norm.pdf(x, mu2[i], sigma2[i])

        # Maximisation:
        pi1[i + 1] = np.mean(resp)
        mu1[i + 1] = np.sum(resp * x) / np.sum(resp)
        mu2[i + 1] = np.sum((1 - resp) * x) / np.sum(1 - resp)
        sigma1[i + 1] = np.sqrt(np.sum(resp * (x - mu1[i + 1]) ** 2) / np.sum(resp))
        sigma2[i + 1] = np.sqrt(
            np.sum((1 - resp) * (x - mu2[i + 1]) ** 2) / np.sum(1 - resp)
        )

    return pi1, mu1, sigma1, mu2, sigma2
```

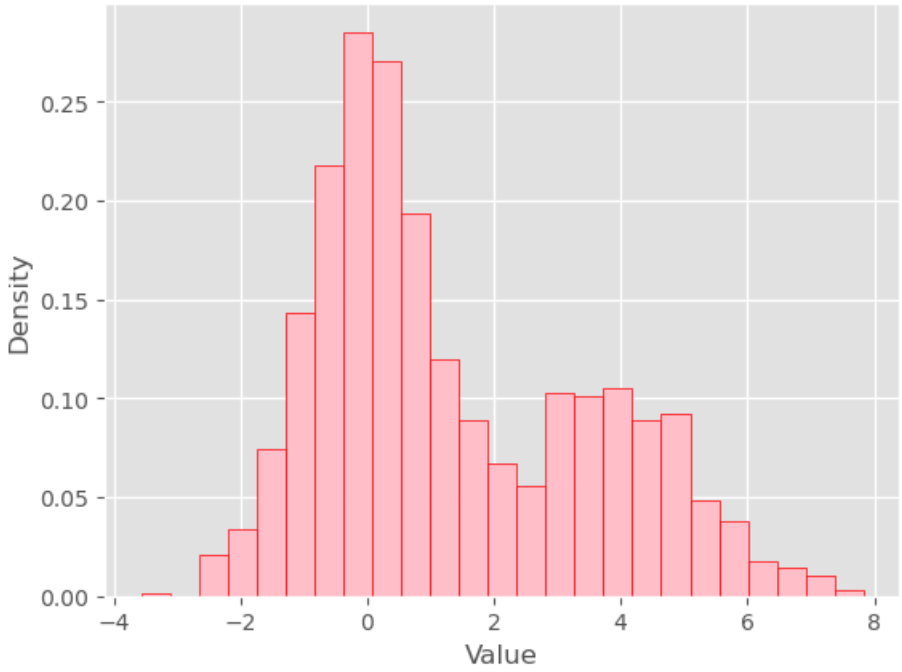
Part (b)

First generate the simulated data, 1000 samples from $N(0, 1)$ and 500 samples from $N(4, 1.5)$:

In [3]:
```
np.random.seed(1234)
x = np.concatenate(
    (np.random.normal(0.0, 1.0, 1000), np.random.normal(4.0, np.sqrt(1.5), 500))
)
```

Draw a histogram:

In [4]:
```
plt.hist(x, bins=25, density=True, color="pink", edgecolor="red")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()
```



Then let us run the EM algorithm:

In [5]:
```
pi1, mu1, sigma1, mu2, sigma2 = gaussian_mixture_em(x, 100)
```

Plot the histories to assess convergence:

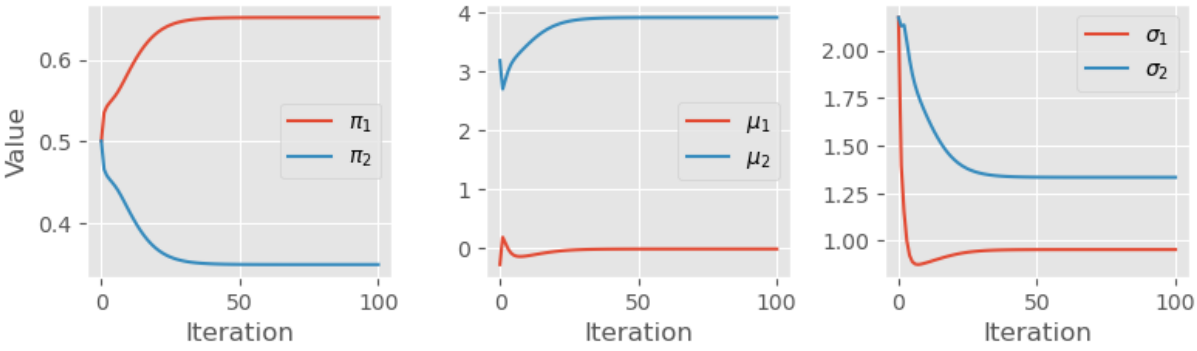
In [6]:
```
fig, axs = plt.subplots(1, 3, figsize=(8, 2.5))

axs[0].plot(pi1, label=r"$\pi_1$")
axs[0].plot(1 - pi1, label=r"$\pi_2$")
axs[0].set_xlabel("Iteration")
axs[0].set_ylabel("Value")
axs[0].legend()

axs[1].plot(mu1, label=r"$\mu_1$")
axs[1].plot(mu2, label=r"$\mu_2$")
axs[1].set_xlabel("Iteration")
axs[1].legend()

axs[2].plot(sigma1, label=r"$\sigma_1$")
axs[2].plot(sigma2, label=r"$\sigma_2$")
axs[2].set_xlabel("Iteration")
axs[2].legend()

plt.tight_layout()
plt.show()
```



Finally, plot the extimated mixture density over the histogram:

In [7]:
```
grid = np.linspace(np.floor(np.min(x)), np.ceil(np.max(x)), 201)
dens = pi1[-1] * norm.pdf(grid, loc=mu1[-1], scale=sigma1[-1])
dens += (1 - pi1[-1]) * norm.pdf(grid, loc=mu2[-1], scale=sigma2[-1])
```

```
plt.hist(x, bins=25, density=True, color="pink", edgecolor="red")
plt.plot(grid, dens, color="navy")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()
```